simplifynext

ignite
simplifynext

AGENTIC AI HACKATHON 2026

Fueling Tomorrow's Changemakers

NUS National University of Singapore | Centre for Future-ready Graduates

NANYANG TECHNOLOGICAL UNIVERSITY SINGAPORE

SMU SINGAPORE MANAGEMENT UNIVERSITY

SUTD SINGAPORE UNIVERSITY OF TECHNOLOGY AND DESIGN

Agenda

1. Tackling the Problem Statement
2. Agent Classes
3. Best Practices
4. Case Studies and Measuring Performance
5. Presentation Prep & Submission Guidelines
6. Q&A

simplifynext

Topic #1

Tackling the Problem Statement

Problem Statement (Recap)

Design for a World in Transformation

Change is everywhere - in how we live, learn, and relate to one another. Transformation takes time, effort, and the right support at the right moment.

This is your chance to build something that helps. We envision a solution that plans, acts, and adapts over time.

Your team will choose the problem and decide who it serves. You will design a solution that thinks ahead, takes action, and leaves people genuinely better off.

Design Thinking: Framing the Problem Statement

Design Thinking

- **Empathise**
  - We will observe and talk to the people we serve
- **Define**
  - We will frame one sharp problem statement
- **Ideate**
  - We will generate many options, then choose one
- **Prototype**
  - We will build the smallest proof of the idea
- **Test**
  - We will put it in front of real users and learn

We will revisit these stages as we learn.

Writing the Problem Statement

- **Use the POV format**
  - [User] needs [a way to …] because [insight].
- **Worked example**
  - A caregiver needs a reliable way to track daily medication because missed doses lead to avoidable hospital visits.
- **Pressure-test it**

✓ Names a real user, a real need, and the evidence

✗ Names a tool, a feature, or a technology

We will agree on one statement first.

Six Ways a Problem Statement Fails

Every one of these reads well in the room and collapses under a judge's first question. Check our own statement against all six before we write any code.

The Solution in Disguise

The statement describes the thing we already decided to build, so the design work has been skipped.

✗ *"Students need an AI chatbot for course advice."*

✓ **Name the person and the moment they are stuck.**

The Everyone Problem

The user is all of humanity, so no design decision follows from it and every feature seems justified.

✗ *"People need better access to mental health support."*

✓ **Choose one person we can picture and describe.**

The Missing Because

A need is asserted with nothing behind it, so we are designing from a feeling.

✗ *"Elderly residents need companionship."*

✓ **Find the evidence, then write the insight it supports.**

The Boiling Ocean

The statement is true, enormous, and beyond what any team can move in four days.

✗ *"Singapore needs a sustainable food supply chain."*

✓ **Cut one slice we can finish and demonstrate.**

The Solved Problem

A mature product already does this well, so the bar for our version sits impossibly high.

✗ *"Commuters need to know when the next bus arrives."*

✓ **Look for what the existing tools still leave undone.**

The Comfortable Guess

Written from the team's imagination, with no contact with anyone who lives the problem.

✗ *"Job seekers need help writing resumes."*

✓ **Talk to two real users this week and rewrite after.**

The fastest check we have: read the statement aloud to someone outside the team. If they ask what we are building, the statement is still doing its job.

Weak to Sharp: Rewriting the Statement

The same idea, written twice. Square brackets mark evidence we still owe, and every bracket needs a source and a date before the statement is finished.

WEAK STATEMENT	WHAT IS MISSING	SHARPER VERSION
✗ *"People need better access to mental health support."*	No named user, no moment, no evidence	✓ A polytechnic student in their first semester needs a way to reach a counsellor the same day, because the wait for a first appointment runs [N weeks, cite source] and distress peaks in the fortnight before exams.
✗ *"Elderly residents need an AI companion robot."*	Names the technology, so the solution is already assumed	✓ An older adult living alone needs someone to notice within the hour that they have fallen, because [cite: share of lasting harm attributed to delay before help arrives].
✗ *"Businesses need to be more efficient with documents."*	The user is a category, and the need is abstract	✓ A claims officer needs the matriculation number and incident date lifted from scanned forms automatically, because keying them by hand takes [N minutes per claim] and the errors surface [N weeks] later.
✗ *"Students need help with career planning."*	True of almost everyone, so it guides no decision	✓ A final-year student two months from graduation needs to see which roles their electives qualify them for, because [cite: proportion graduating into unrelated work].

Each sharper version names one person, one moment, and one piece of evidence.

Pressure-Testing Our Own Statement

put our statement through these five questions as a team. Any answer of no sends us back to Empathise before we write code.

1	**Can we name one person?**	A role at a moment, specific enough that we could picture them walking into the room. 'Users' and 'people' will not pass.
2	**Can we cite the evidence?**	A figure, a source, and a date. If the only support we have is that it feels true, we have an assumption to go and test.
3	**Would that person recognise themselves?**	We have spoken with at least one of them. A statement written entirely from our own imagination ends up describing the team.
4	**Does it survive a different solution?**	The statement should still hold if another team builds something completely unlike ours. It describes the problem, and it belongs to no design.

Question 4 is the sharpest one. A statement that only makes sense once we describe our own build has quietly become a product pitch, and rewrite it before going further.

Where Agentic AI Belongs in the Story

We should keep the problem statement free of technology, but justify agentic AI as well – would this be possible without agentic AI? Both points are graded, and they answer different questions.

THE PROBLEM STATEMENT

Describes a person, a need and an insight, in language the person themselves would use.

- Names a role at a specific moment
- Carries evidence with a source
- Stays true whatever anyone builds
- Reads as plainly as a sentence spoken aloud

"A claims officer needs the policy number lifted from scanned forms, because keying them by hand takes [N minutes] per claim."

THE SOLUTION OVERVIEW

Describes what we built and argues why an agentic approach earns its place in that problem.

- Names the planning, acting and adapting
- Explains what a fixed workflow would miss
- Shows the reasoning a judge can follow
- Connects each capability to the person above

"Our agent reads the form, checks the policy system, and escalates only the claims where the two disagree."

The test that separates them: Would this problem still exist if agentic AI had never been invented?
A yes means we have written a problem statement. A no means we have written a solution, so go back to the person and start again.

simplifynext

Topic #2

Agent Classes

Digital AI Agent Classes

We see AI agents as falling into **broad classes**, depending on **the main problem they solve** and **how they interact with users/systems**. Many agents will belong to multiple classes. These are just examples to get you started.

Information
Answer & Advise

Agents that **retrieve, summarize, or explain** information.

Extraction
Parse & Transform

Agents that **convert** unstructured content into **structured, actionable data**.

Transaction
Do & Automate

Agents that **perform tasks on behalf of the user** by integrating with systems.

Decision-Support
Guide & Recommend

Agents that **help with choices** by analyzing data and recommending actions.

Creative/Generative
Create & Draft

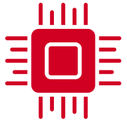
Agents that **produce new content** based on user intent.

Orchestration
Coordinate & Integrate

Agents that **combine multiple systems and flows**, acting as a "control tower.

Personalized
Adapt & Learn

Agents that **customize responses** based on semantic and episodic context.



Embedded
Live Where People Work

Agents that **exist inside other apps or processes** rather than as standalone chatbots.

Physical AI Agent Classes

We group physical AI agents into broad classes by what they sense, what they change in the world, and how they work alongside people. We expect most real deployments to span several classes at once.

SENSE

Perception

Sense & Interpret

Turn raw sensor streams from camera, LiDAR, radar and IMU into a structured read of the physical world.

Note : Perception belongs here when the reading feeds a decision that changes physical state. Analysing recorded media for a report sits with Extraction Agents on the digital side.

SENSE

Monitoring & Inspection

Watch & Flag

Keep persistent watch over an asset or an area, detect drift and anomalies, and raise them.

ACT

Navigation

Where To Go

Build a map, plan a route, and avoid obstacles that move.

ACT

Locomotion & Control

How To Move

Govern gait, balance and force so the body stays upright over unfamiliar terrain.

ACT

Embodied Task

What To Do

Decompose a goal into skill calls, then replan when the world resists.

BOTH

Human-Robot Interaction Agents

Work Alongside People

Take instruction by voice or gesture, predict intent, and yield safely around people.

NOT RECOMMENDED FOR THIS HACKATHON

BOTH

Fleet Coordination Agents

Many Machines, One Goal

Allocate tasks, manage shared space, and schedule charging across many robots.

NOT RECOMMENDED FOR THIS HACKATHON

SENSE Perceives only

ACT Acts on the world

BOTH Does both

Teams will focus on the six classes in scope. We will cover the greyed classes for awareness only.

What Changes When an Agent Has a Body

We ask the same three questions physically

Where To Go *Navigation*

Build a map, plan a route, and avoid what moves.

What To Do *Task*

Break a goal into skills and replan on failure.

How To Move *Control*

Govern gait, balance and force on unfamiliar ground.

Plan for four physical constraints

1. **Account for latency**

   A control loop that misses its window produces a fall. Budget compute against the loop rate.

2. **The world is partly visible**

   Sensors stay noisy and occluded, and they disagree with each other. Fuse and arbitrate between them.

3. **Hardware failure is routine**

   Treat battery drain, thermal throttling and comms dropout as normal operating cases to be handled in code.

4. **Actions are final**

   A dropped object stays dropped and a collision stays collided. Design approvals and stop conditions to match.

simplifynext

Topic #3

Best Practices

The Agentic AI Stack We Teach

A request travels down the layers and a response returns up. We will build on this stack so what you learn from the technical sessions transfers into the submission.

Request ↓ **Response** ↑

INTERFACE
flexible

- **Your own front end** — no framework is required

DEPLOY & SERVE

- **AgentCore Runtime** — @app.entrypoint handler
- **Local first** — POST to localhost:8080

ORCHESTRATE

- **create_react_agent** — the ReAct loop, prebuilt
- **LangGraph** — nodes, edges, routers, loops
- **deepagents** — planning, files, sub-agents
- **Claude Agent SDK** — MCP tools, files, bash

SCHEMA

- **Pydantic schema** — descriptions are the prompt

ACCESS

- **boto3 InvokeModel** — raw request and response
- **Converse API** — unified message shape
- **ChatBedrockConverse** — the LangChain wrapper

MODEL

- **Claude Haiku 4.5** — global profile, our default
- **Claude Sonnet 4.5** — single-region alternative

CROSS-CUTTING

- Typed state with reducers where nodes write concurrently
- InjectedState reaches large objects without a prompt
- InMemorySaver plus thread_id carries a conversation
- Token usage returns on every response
- The @app.entrypoint handler is the seam a UI calls

GUARDRAILS & LIMITS

- Verify model access in the region we deploy to
- Read model ids from a constant, never build them.
- allowed_tools is an allow-list and a security boundary
- Bound every loop with a counter held in state

Building Agents That Hold Up

Most agent demos work at five turns and fall apart at fifty. These four decisions are made before we write code, and they are what separates a shipped agent from a demonstrated one.

01 Context rot is real

Everything the model can see shares one window. Well before the limit, accuracy and consistency degrade while cost and latency rise on every turn of the loop.

Build short, single-purpose agents that do one job and exit.

02 Bound every loop

A refine loop that exits when the critic is satisfied will sometimes never be satisfied, and we discover it from the bill.

Keep a hard iteration cap in state that ignores the model's judgement.

03 Descriptions are the interface

The model chooses tools and sub-agents by reading names, descriptions and parameter docs. A vague description produces an unused tool or a badly called one.

Ttreat tool descriptions as the highest-leverage prompt text we write.

04 Keep payloads small

Tools that return page dumps fill the window with material the model has to re-read on every subsequent turn.

Return small typed results and hold large objects in state instead.

Frameworks now ship help for the first item. A summarisation middleware watches the window and compresses old material once it fills, so we get context management as a constructor argument.

Development Best Practices

simplifynext

- **Code Readability:**
  - Write clean, commented code. Use clear variable names and consistent formatting. (usual best practices)
  - Organize code to showcase application of Agentic AI application & techniques

- **Project Folder Structure:**
  - Create a logical folder hierarchy. A good structure might include src/, docs/, data/, and tests/.

- **Documentation:**
  - Keep it concise and clean.
  - A good README.md file is sufficient for judges to understand your project.

- **Error Handling**
  - Baseline: error handling for functional resilience
  - Going further: How can you make it actionable for business users?

Development Best Practices

simplifynext

Testing

- **Accuracy Testing**
  - Understand the difference in nuance compared to traditional ML accuracy measurements – e.g. Precision/Recall, F-1, etc
  - Example – For a chatbot use case, how would you prepare test data and measure accuracy?
  - Are Agentic AI results always Yes/No?

- **Evaluation Metrics**
  - Identify relevant data points to be captured, in translation to the Problem Statement & Solution Objective
  - Justify your testing methodology

simplifynext

Topic #4

Case Studies

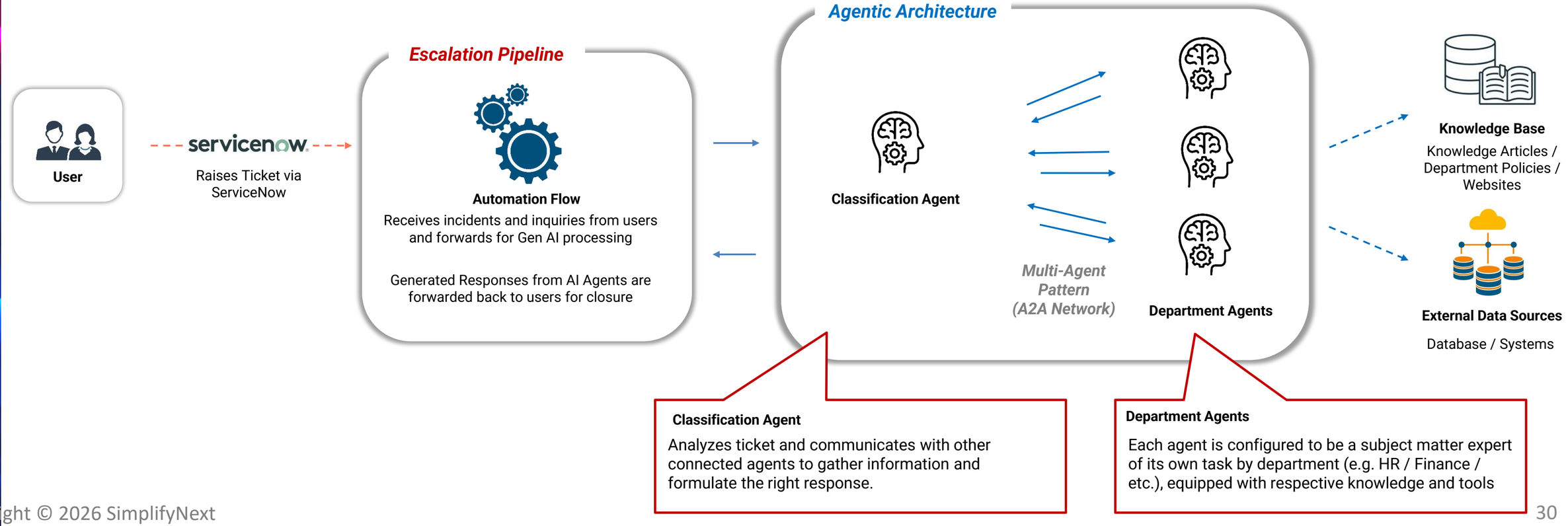
Case Study 1:
Educational Support Hub

simplifynext

Problem Statement:

- Human agents are currently spending substantial effort on routine, non-complex tickets from Students, Faculty, and Staff, reducing their capacity to handle complex incidents.
- At peak period, this university saw approx. 100 tickets per day for the human agents to process.

Agentic AI Design:



Case Study 2:
Intelligent Exam Generation

simplifynext

Problem Statement:

- Educational institutions and trainers need to design high-quality, curriculum-aligned assessments quickly, but face:

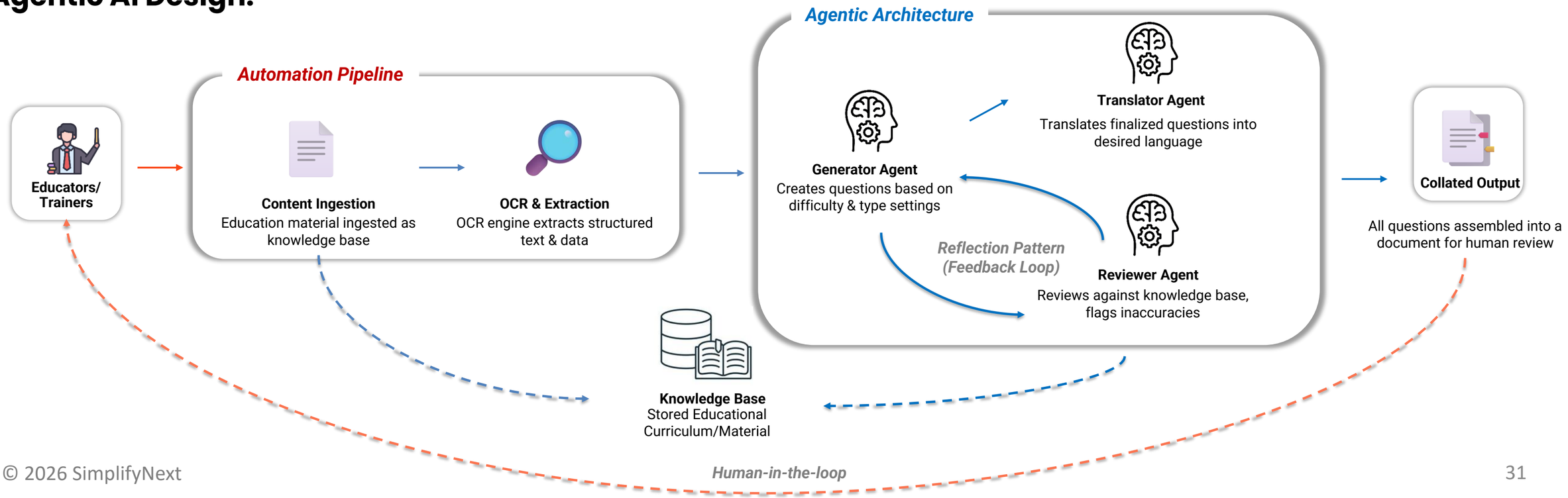
Time Constraints
Manual exam creation takes weeks of effort.

Content Consistency
Ensuring alignment with curriculum and learning outcomes is tedious.

Variety of Question Types
Exams require MCQs, essays, case-based, and scenario-based questions.

Ineffective Traditional Methods
Manual drafting and peer review are slow and prone to inconsistencies.

Agentic AI Design:



Measuring Performance of Digital AI Agents

Some examples of measuring performance:

01

Schema Validation Pass Rate

Measure the share of outputs that parse and validate on the first attempt.

“Is the output usable by another system?”

02

Tool-Call Success Rate

Count the share of tool calls that return a usable result, and log the ones that fail.

“Does the agent reach for the right hands?”

03

Task Completion Rate

Measure requests resolved end to end without a human stepping in to finish them.

“Did it carry the job to the end?”

04

Token Cost Per Run

Sum input, output, cache read and cache creation tokens across a whole run.

“What does one answer actually cost?”

05

Loop Discipline

Record iterations per task against the cap, and how often a run reaches that cap.

“Is it converging or circling?”

06

Answer Fidelity

Score against a reviewed ground-truth set, using a rubric or an LLM judge.

“Is it right, as well as plausible?”

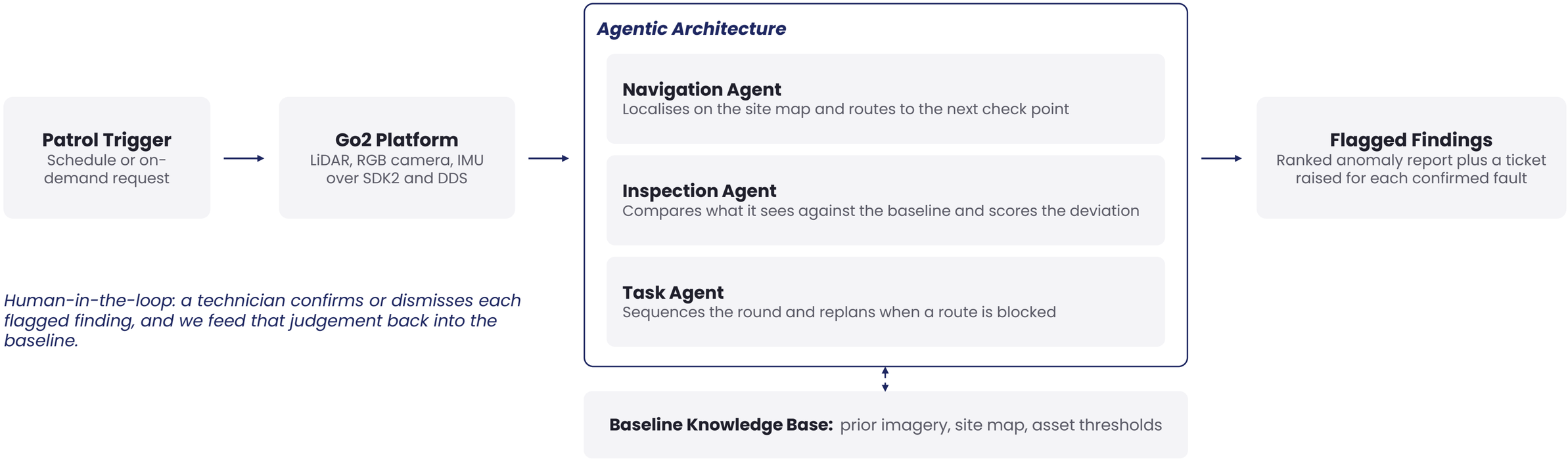
Case Study 3:

Autonomous Facility Inspection

Problem Statement:

- Facility teams walk fixed inspection routes on a schedule, so a fault that appears between rounds stays unseen until the next walk.
- A campus round covers dozens of check points across several buildings and consumes hours of technician time that we would rather spend on repair.
- We want continuous coverage, and we want the agent to decide what deserves a human look.

Agentic AI Design:



Human-in-the-loop: a technician confirms or dismisses each flagged finding, and we feed that judgement back into the baseline.

Classes applied: Perception | Monitoring & Inspection | Navigation | Embodied Task

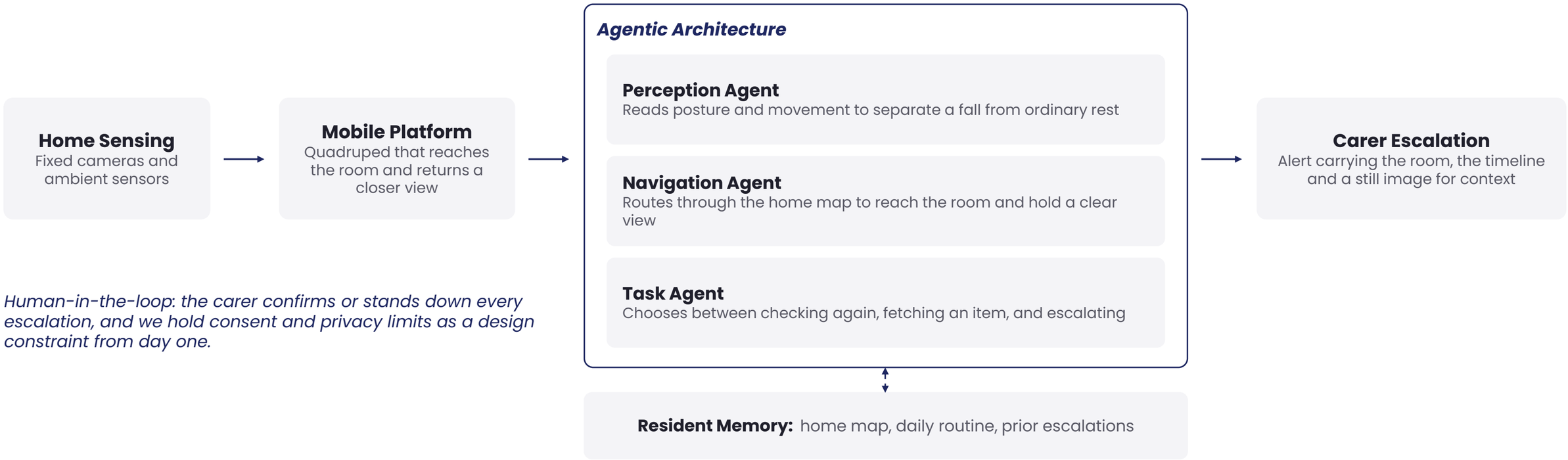
Case Study 4:

Assistive Ageing-in-Place Companion

Problem Statement:

- Older adults living alone face long delays between a fall and the arrival of help, and that delay drives much of the eventual harm.
- Wearable alert buttons depend on the resident wearing the device and staying conscious enough to press it.
- We want an agent that observes the home continuously, recognises when something has changed, and escalates to a carer with useful context.

Agentic AI Design:



Human-in-the-loop: the carer confirms or stands down every escalation, and we hold consent and privacy limits as a design constraint from day one.

Classes applied: Perception | Monitoring & Inspection | Navigation | Embodied Task

Measuring Performance of Physical AI

Some examples of measuring performance:

01

Task Success Rate

Measure the share of attempts completed end to end on the real target.

“Did it finish the job?”

02

Intervention Rate

Count human takeovers per run or per hour. Treat this as the honest measure of autonomy.

“How often did a person rescue it?”

03

Safety Record

:og collisions, near misses and emergency stops, and we hold this line while tuning for speed.

“Is it safe to stand next to?”

04

Cycle Time

Time each completed task, including recovery from the agent's own mistakes.

“Is it fast enough to be worth using?”

05

Robustness

Re-test under changed lighting, terrain, clutter and starting position.

“Does it hold when conditions shift?”

simplifynext

Topic #4

Understanding the Deliverables

Understanding the Deliverables

The following are the required deliverables:

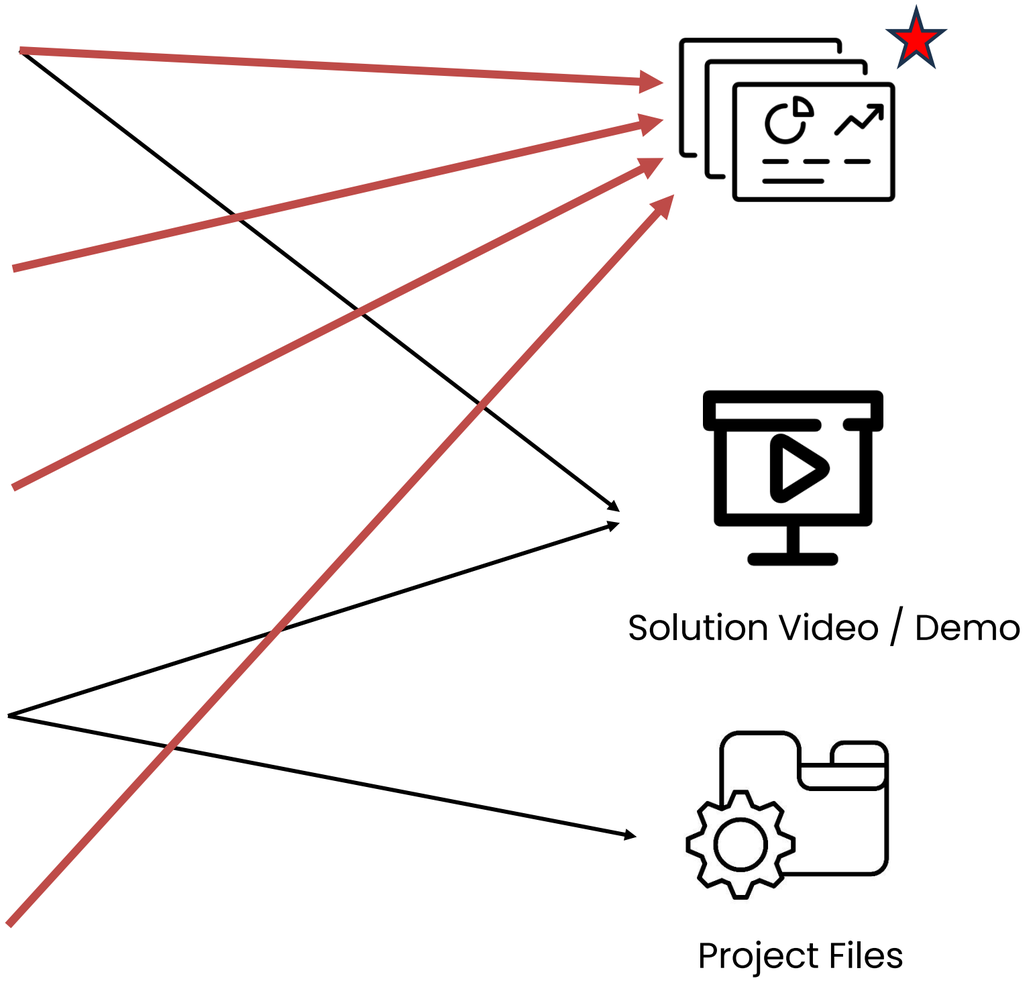
- Project Files / Workflow (Max 5gb; 1 submission only)
- **Presentation Deck (Max: 10 slides)**
- **Digital Solution Video (Max: 5 minutes)**
- **OR Video Recording of Simulation (Max: 5 minutes)**

Your deliverables should:

- Answer the Question;
  - clearly explain the problem and your innovative solution.
- Showcase your knowledge of Agentic AI,
  - technical soundness and functionality.
- Create Business Impact:
  - Demonstrate that the solution addresses the problem statement.

Judging Criteria Alignment

Benefits delivered by the solution	Clearly link your features to measurable improvements.
Original / Innovative idea	Point out what makes your approach different from existing solutions.
Effectiveness of the Solution	Use evidence (data, tests, scenarios) to prove it works.
Technical Quality and Superiority of Solution	Show a functional prototype and explain how it is robust.
Presentation	Structure your story well, rehearse delivery.



Judging Criteria (1/3)

Criteria	Weight	Description	Scoring
Benefits delivered by the solution	20%	What positive impact does the solution bring to users, the organization, or the community? *Benefits may include increased revenue, productivity, quality, compliance, or quality of life.*	**2 points** Clear benefits; scalable or easily adopted **1 point** Clear benefits **0 point** No or limited benefits
Original / Innovative idea	20%	How original and creative is this solution?	**2 points** Unique and innovative approach to the problem **1 point** Based on existing ideas but addresses the problem **0 point** Not unique; existing solutions address it effectively

Judging Criteria (2/3)

Criteria	Weight	Description	Scoring
Effectiveness of the Solution	20%	How effective is the solution in addressing the problem or opportunity?	**2 points** Fully addresses and resolves the problem **1 point** Partially addresses the problem, but not fully resolved **0 point** Minimal effectiveness in solving the problem
Technical Quality and Superiority of Solution	20%	Is the prototype functional and technically sound?	**2 points** Technically advanced, fully functional prototype with minimal work needed for production **1 point** Functional prototype with minor work needed for production **0 point** Partially functional or does not meet core requirements

Judging Criteria (3/3)

Criteria	Weight	Description	Scoring
Presentation	20%	How effective is the team in articulating the problem statement and explaining how the solution tackles the problem and delivers the benefits?	**2 points** Clearly explained the problem and demonstrated the solution's benefits **1 point** Partially explained with some prompting or clarification needed **0 point** Unable to clearly explain the problem or demonstrate benefits

Project Files – Key Points

- Build a simple **proof-of-concept** to demonstrate your Agentic AI component.

- The Details:
  1. Keep documentation concise. A good README will suffice, covering
     1. Instructions to run your code
     2. Overview of your code, and purpose of each script/file

  2. Environment setup
     1. Virtual Environments. Provide requirements.txt. Docker setup is also acceptable.
     2. Path Variables
     3. Secrets and Keys. Use .env files

  3. Language/Stack
     1. **Python** is strongly recommended

  4. Execution. No extensive test data is required. What we will look at:
     1. Whether solution can run as demonstrated in video.
     2. If presentation methodology is reflective at code level. In-line documentation where relevant would be helpful.
     3. Testing/Evaluation should be covered in your slides.

Slide Deck: Structure Overview

Sample 10-slide flow:

1. Title & Team - Names, roles, and a one-line mission statement
2. Problem Statement/ Why It Matters -The real-world issue you're tackling, backed by data & why solving this matters for the public good
3. **Solution Overview** - What your agentic AI does and why it's different
4. **Methodology** – Functional overview of your solution
5. **Technical Architecture** - High-level system design, components, tech stack
6. Innovation & Uniqueness - How your approach stands out
7. **Benefits Delivered** - Quantified improvements or advantages
8. Demo Preview - Screenshots or flow of your live/recorded demo or simulation
9. **Roadmap & Future Potential** - Where the solution can go next
10. Conclusion & Call to Action - Recap and inspire adoption

Slide Deck: Key Content Tips

- Keep 1 core message per slide, avoid overcrowding with too much info.
- Use visuals where relevant (diagrams, icons, infographics).
- Use consistent branding: colors, fonts, and style.
- Tie each slide to judging criteria (impact, originality, effectiveness, technical quality, presentation).
- Avoid generic claims; use data or real examples to prove points.
  - Example: Instead of 'Improves efficiency', say 'Reduces processing time by 30%'.

Slide Deck: Example

- Problem & Impact – What to Include?
  - Clear, concise problem statement
  - Evidence: Data, statistics, or credible reports showing urgency.
  - A relatable user story or persona - Who is affected and how?
  - Why solving this is important?

- Let's take a look at some examples – DOs vs DON'Ts
  - "1 in 4 adults experience mental health issues annually, but only 40% receive treatment." – *Clear Statement*
  - "WHO reports depression costs the global economy $1 trillion yearly in lost productivity." – *Credible Data*
  - "Meet Jane, a university student who struggles to access timely support." – *Show User Story*
  - "Lack of early intervention leads to worsening conditions and higher healthcare costs." – *Connect to Public Good*
  - "Mental health is a big problem everywhere." – *Too Vague*
  - "This issue affects everyone in the world." – *Too Broad, No Focus*
  - "Our solution will change the world." – *Buzzwords, No Evidence*

Slide Deck: Example

- Solution & Technical Flow – What to Include?
  - Short, simple description of what your solution does.
  - Agentic capabilities: planning, acting, adapting
  - Diagram of how the system works (inputs → reasoning → actions → feedback)
  - Key tech stack (tools, languages, AI models).

- Let's take a look at some examples – DOs vs DON'Ts
  - Plain language: "Our AI agent detects early signs of stress from daily mood check-ins."
  - Show flow: Inputs (user check-ins, wearable data) → AI reasoning (detect patterns) → Actions (recommend breathing exercises, alert counselor) → Feedback (track improvements).
  - Agentic features: "Plans personalized activities, adapts based on user responses."
  - Keep tech stack relevant: "LLM for reasoning, mobile app for user interface, Python APIs for data ingestion."
  - "We built a GPT-4 + RLHF + custom neural net…" (jargon-heavy, no context).
  - Only listing features: "Chatbot + API + dashboard" (no user benefit).
  - No diagram/flow, making it hard to visualize.
  - Forgetting to link how features actually improve access to care.

Demo Video: Structure

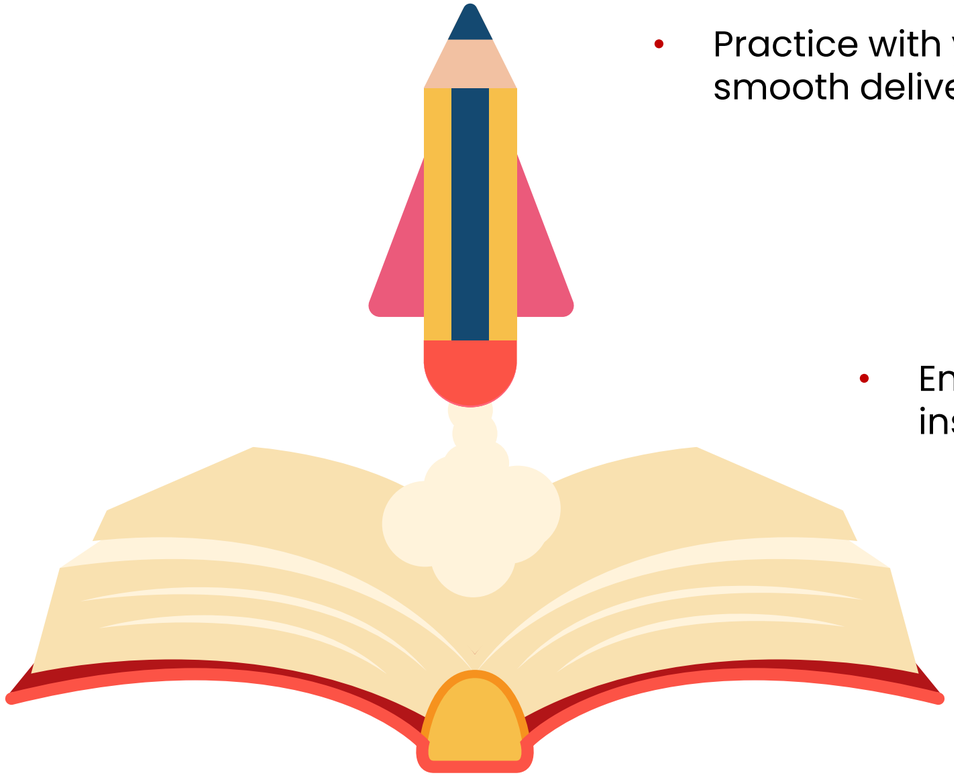
- Suggested 5-minute breakdown:
  1. Opening hook (0:00–0:30) – Start with a relatable story or bold fact.
  2. Problem explanation (0:30–1:00) – Show the problem in action.
  3. Solution overview (1:00–1:30) – State how your solution addresses the issue.
  4. Live/recorded demo (1:30–3:30) – Walk through features, show it working.
  5. Impact & benefits (3:30–4:15) – Show before/after or metrics.
  6. Closing & call to action (4:15–5:00) – End with a vision, invite adoption.

Demo Video: Structure

- Show, don’t just tell – demonstrate the AI making a decision or taking action.
- Highlight agentic features: where does it plan, act, adapt?
- Use clear voiceover or captions for accessibility.
- Avoid jargon or explain terms briefly.
- Pace your delivery – 5 minutes is short, plan your time for each section.
- Example: If showing an AI chatbot, don’t just show the interface – narrate the reasoning process behind its responses.

Final Tips for Successful Presentation

- Focus on a strong problem statement within reach
- Make the value obvious to judges in the first minute.
- Practice with your team to ensure smooth delivery within time limits.



- Keep a logical flow – problem → solution → impact.
- End with a powerful call to action – inspire judges to believe in your idea.